Future Generation Computer Systems 56 (2016) 623–639

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

Mobile crowdsensing as a service: A platform for applications on top of sensing Clouds

Giovanni Merlino [a,b], Stamatis Arkoulis [c], Salvatore Distefano [d,e,*], Chrysa Papagianni [c], Antonio Puliafito [a], Symeon Papavassiliou [c]

[a] Dip. di Ingegneria (DICIEAMA), Università di Messina, Italy
[b] Dip. di Ingegneria (DIEEI), Università di Catania, Italy
[c] School of Electrical and Computer Engineering, National Technical University of Athens, Greece
[d] Social and Urban Computing Group, Higher Institute of Information Technology and Information Systems (ITIS), Kazan Federal University, Russia
[e] Dip. di Elettronica, Informazione e Bioingegneria (DEIB), Politecnico di Milano, Italy

HIGHLIGHTS

- Emulation of deployment comparing (i) prototype of MCSaaS and (ii) current practices.
- Modeling of MCSaaS paradigm by Petri nets and evaluation against conventional MCS.
- Description of architectural elements of the platform and their interactions.
- MCSaaS platform for mass deployment of MCS apps, and their elastic user base adaptation.
- Mobile crowdsensing as a service paradigm for MCS apps design and deployment.

ARTICLE INFO

Article history:
Received 22 November 2014
Received in revised form
11 September 2015
Accepted 12 September 2015
Available online 9 October 2015

Keywords:
Cloud
IoT
Infrastructure as a Service
Volunteer contribution
Sensors and actuators
Runtime customization

ABSTRACT

Consumer-centric mobile devices, such as smartphones, are an emerging category of devices at the edge of the Internet. Leveraging volunteers and their mobiles as a (sensing) data collection outlet is known as Mobile Crowd Sensing (MCS) and poses interesting challenges, with particular regard to the management of sensing resource contributors, dealing with their subscription, random and unpredictable join and leave, and node churn.

To facilitate and expedite the (commercial) exploitation of this trend, in this paper we propose to adopt a service-oriented approach to cope with MCS application deployment into a sensing Cloud infrastructure, decoupling the MCS application domain from the infrastructure one. To this purpose we provide the building blocks for implementing such a novel take on MCS, which from a Cloud layering perspective can be identified as a platform service, i.e., an MCS as a service (MCSaaS). A prototype implementation that serves as a blueprint and a proof-of-concept of the proposed framework is presented, while an evaluation of the effectiveness of the MCSaaS paradigm has been provided using suitable mobility-related use cases for a validation of the concept, as well as a modeling approach through the adoption of generalized stochastic Petri nets.

1. Introduction and motivations

Current trends, with specific regard to cyber physical systems and Internet of Things (IoT), suggest that one of the most interesting thrusts towards pervasive services comes from opportunistic and participatory sensing paradigms, such as Mobile Crowd Sensing (MCS). MCS leverages the pervasiveness of smartphones and other portable devices, enabling users and community groups to collectively share data from onboard sensing resources so as to measure phenomena of common interest, exploiting social dynamics. The contributor has also the possibility to augment raw data with context as metadata. This community-driven sensing trend is brought about by machine interactions at different levels, including data communications, collection, processing and mining. Commencing crowd-sourcing and sensing duties from mobiles, involving device owners as volunteering participants, potentially renders end users both contributors and consumers of large volumes of (curated) data.

* Corresponding author at: Dip. di Elettronica, Informazione e Bioingegneria (DEIB), Politecnico di Milano, Italy.
E-mail addresses: gmerlino@unime.it, giovanni.merlino@dieei.unict.it (G. Merlino), stark@cn.ntua.gr (S. Arkoulis), s_distefano@it.kfu.ru, salvatore.distefano@polimi.it (S. Distefano), chrisap@noc.ntua.gr (C. Papagianni), apuliafito@unime.it (A. Puliafito), papavass@mail.ntua.gr (S. Papavassiliou).

http://dx.doi.org/10.1016/j.future.2015.09.017
0167-739X/

However, there are several key issues that need to be addressed for the MCS paradigm to experience widespread adoption [1,2]. Firstly, a unified architecture for supporting MCS applications is required to enable reusability of software components, facilitating shorter time to market cycles. Existing MCS applications are built as stand-alone ones, while common challenges, e.g., related to resource engagement, get independently revisited each time, or are not addressed at all. The heterogeneity of mobile Operating Systems (OS) and sensor hardware further amplifies the problem. As a result sensing and processing activities usually result in carrying out similar tasks (i) within a single device (contributing node) for different MCS applications, resulting in energy starvation and (ii) across multiple, neighboring devices, leading to spikes in bandwidth usage and processing requirements at the back end. Both cases highlight a non-scalable deployment model.

Furthermore, MCS applications rely on every single contributor for local deployment duties. While most MCS apps require a critical mass of contributors to be deemed useful, app adoption is bounded by the rate at which users keep track of, and install, newly introduced ones. A substantially low rate as, according to recent reports, e.g., in 2014 nearly 80% of the 1.2 million apps available at the Apple App Store had hardly any downloads at all.[1] Providing resources on a volunteer basis is one of the foremost limitations to the large scale exploitation of the MCS paradigm, as it naturally bears constraints related to contribution churn. In terms of MCS applications this sets the need for "marketing" strategies aimed at increasing the number of subscriptions, stimulating, retaining and rewarding potential contributors through incentives. However, even if the enrollment activities (or even mechanisms, such as credit-reward systems) may be highly effective, the subscription process is usually characterized by long and smooth dynamics. Therefore a significant time delay may be experienced before getting a significant stream of sensing data. This issue may strongly confine the scope of the MCS paradigm to a shortlist of applications featuring broad, long-established supporting communities of potential contributors.

From a different perspective, another significant trend moves IoT towards service-oriented and Cloud computing paradigms [3–5]. In this view, the Cloud is not just a technology to support the archiving of sensed data coming from pervasive IoT infrastructure, but also a model and a paradigm to adopt in the management of the underlying resources and things.

Following this IoT-Cloud research trajectory, in this paper, a novel platform for opportunistic (mass) exploitation of contributed resources for MCS apps is presented to get more insights as to whether the MCS paradigm may indeed be applied at large-scale in the IoT context. The proposed approach overcomes several of the aforementioned hurdles, by facilitating what is essentially the most difficult endeavor for prospective MCS entrepreneurs, i.e., offering a level playground, with homogeneous access to wildly different underlying ecosystems. To this end, we identify two classes of concerns with regard to (unassisted) dissemination of MCS apps; (i) *infrastructure*-related, mostly focused on mechanisms to enroll and manage voluntarily contributing nodes, as well as to abstract sensing resources and enable uniform access, and (ii) *application*-related, mainly devoted to mechanisms for interfacing with enabled infrastructure, asking for the required sensing resources and, once obtained, deploying the MCS app onto the resource-hosting nodes. This way we are able to decouple infrastructure resources (*supply*) from application requirements (*demand*) as in Cloud contexts, making development, deployment and operation fully independent.

For delivering this novel MCS approach, the Sensing and Actuation as a Service (SAaaS) framework, proposed in [6], is adopted as the lower-level (infrastructure domain) enabler. SAaaS is based on the service-oriented (and Cloud-inspired) approach of elastically providing (virtual) sensing and actuation resources on demand, gathered from underlying (contributed) physical nodes. The study at hand extends and adapts the SAaaS paradigm for enabling rapid MCS app deployment and streamlining their operation, actually providing an *MCS as a Service* (MCSaaS) platform.

There are a number of advantages in dealing with MCS from the (SA)aaS angle. Virtualizing and customizing sensing resources, starting from the capabilities provided by the SAaaS model, allows for concurrent exploitation of pools of devices by several platform/application providers. Delivering MCSaaS may further simplify the provisioning of sensing and processing activities within a device or across a pool of devices. Moreover, decoupling the MCS application from the infrastructure promotes the roles of a sensing infrastructure provider that enrolls and manages contributing node(s), below, and of a platform provider on top of that, acting as a broker between (i) the former and (ii) the MCS application provider, enabling the latter to focus on the application and leave concerns and enforcement about requirements (type of resources, availability, etc.) to the platform.

In this paper we provide a high level overview of the proposed MCSaaS approach complemented by a description of the distinct architectural elements and their interactions. Moreover we highlight the benefits of the proposed framework as opposed to the conventional MCS approach, by means of (i) a prototype implementation of the MCSaaS framework under the guise of the MobiCSOS stack and the emulation of a real-world application deployment; and by (ii) modeling with generalized stochastic Petri nets (GSPN) [7].

The main contributions of this work can be therefore identified in:

- a novel (Cloud-based) service-oriented paradigm for MCS, i.e., MCSaaS;
- a reference architecture for the MCSaaS stack;
- a prototype MCSaaS implementation, i.e., MobiCSOS, developing basic mechanisms, such as churn management, that serves as a proof of concept for the proposed approach;
- a preliminary evaluation of the MCSaaS potential on Android mobiles, based on simple real-world scenario and a larger-scale analytical model.

The remainder of the paper is organized as follows. Section 2 provides an overview of related work. Section 3 focuses on the overall MCS scenario, characterizing and motivating the MCSaaS approach we propose, while Section 4 presents the infrastructure elements. Section 5 deals with the main features of the MCSaaS platform, Section 6 addresses the problem of application setup and deployment, while Section 7 focuses on the implementation details of MobiCSOS. The MCSaaS-assisted deployment of MCS apps, with regards to urban mobility use cases, are discussed in Section 8, with some numerical evaluation based both on an emulated scenario and on a GSPN model, while conclusions are drawn in Section 9.

2. Preliminary concepts and related work

2.1. Mobile crowd-sensing

MCS [1] is an emerging trend, lying at the intersection of volunteer and crowd-based computing, IoT, and sensing paradigms. It refers to a broad range of community-powered sensing approaches belonging to either *participatory* [8] and/or *opportunistic* [9] categories, aiming at involving large population of contributors [2,10].

[1] https://www.adjust.com/assets/downloads/AppleAppStore_Report2014.pdf.

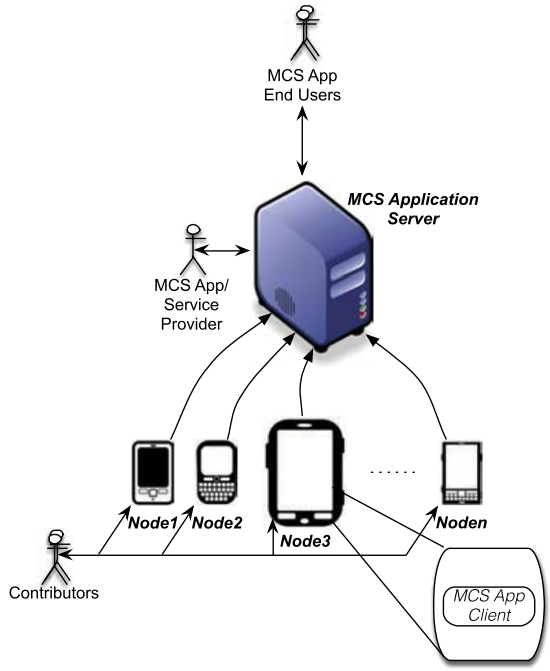


Fig. 1. MCS application scenario.

Participatory sensing is characterized by individuals that are actively involved in contributing to a given application by mean of devices or (meta-)data (e.g., taking a picture), while opportunistic sensing is more autonomous and user involvement is minimal (e.g., continuous sampling of geo-localized data).

A broad range of applications, from mining of urban dynamics [11], to public safety [12], traffic and environment monitoring [13,14], smart lighting [15] and smart cities [16], just to name a few, may be implemented adopting the MCS paradigm. Existing MCS applications are comprised of two main components [1]:

(i) device-specific ones, for data collection, execution of local analytics if needed, and data dissemination,
(ii) the backend, for extensive data analysis, storage and visualization duties.

A simple and generic MCS application scenario is shown in Fig. 1, where the main elements and stakeholders involved in the MCS system are identified based on [1]. These include the contributing *nodes*, e.g., smartphones, tablets and, PDAs, shared by their *Owners* or *Contributors* to build up the *sensing infrastructure environment*, as well as the *Application Service Provider* (MCS ASP), that manages and supervises the whole process, gathering and processing sensed data through the application server, which also interfaces with the *End User* of the MCS application.

2.2. Related work

Although MCS is a quite new and emerging topic, several works deal with related issues. Earliest works on the topic present and characterize the approach in terms of opportunistic [9], participatory [8], social and crowd sensing [17] paradigms. All such concepts have been then grouped under the MCS umbrella [1].

So far, several MCS applications [18,19,13,20] have been developed in different contexts, demonstrating the MCS paradigm is useful in applications directly and indirectly involving different stakeholders and huge populations of users. This has drawn the attention of both the academic and business communities, which, on one hand, started developing new middlewares implementing mechanisms and tools for MCS system management [21,22] while, on the other, are investigating potential exploitations of the MCS approach both in scientific applications and in commercial ones. Therefore, the current state of affairs highlights the need for suitable methodologies and techniques able to bring order in the MCS field, adopting engineering practices and tools to explore the untapped potential of the paradigm.

With specific regard to the issues raised in the previous section, several frameworks have been proposed to support expedite MCS application development and deployment. AnonySense [23] is a privacy-aware framework for opportunistic and participatory sensing. Applications specify the sensing task behavior using a Domain Specific Language (AnonyTL) and then submit it to the AnonySense components and mobile nodes. A polling model is used for task distribution. Downloaded tasks are matched to the nodes based on their context.

Medusa [24] is a programming framework that specifies the workflow of sensing tasks to be executed in smartphones and onto the Cloud. It defines the Med-Script programming language that provides high-level abstractions for the various stages in crowd-sensing tasks as well as for flow control. Moreover it provides a distributed runtime between the Cloud and smartphones.

Pogo [25] proposes a middleware for building large scale sensing testbeds using mobile phones. Both researchers and device owners, each category running the middleware differently, depending on their role. Pogo relies on the XMPP protocol for communication between nodes. Experiments are written in JavaScript.

Vita [26] supports the development, deployment and management of multiple MCS applications/tasks. It consists of both a mobile and a Cloud platform. The former provides the appropriate services (e.g., map and location) that enable the execution of sensing tasks. It optimizes the allocation of a task to a group of users or Cloud servers by using Genetic Algorithms and K-means clustering. The Cloud platform streamlines the development and deployment of MCS applications, integrates and stores uploaded sensing data, as well as metrics related to system operation.

PRISM [27] is also a platform for community-sensing applications that allows the deployment of binaries ready for execution onto mobiles. Method call interposition is used to sandbox-untrusted applications to ensure security and privacy. PRISM follows a push-based model for the automatic deployment of applications to an appropriate set of phones. Efficient tracking of mobiles is implemented mitigating privacy risks and reducing communication overhead.

Most of the aforementioned frameworks propose custom languages for hardware and OS abstraction of the contributing devices or specify ad-hoc sandboxed environments for secure execution of applications. Although convenient, the fact that only pre-programmed functions and software modules are provided to developers results in a less flexible and not as powerful application development environment. In fact, only PRISM [27] allows for native MCS applications deployment (although sensors can only be accessed through a sandbox), while also providing a mechanism for selection of contributing devices. By not spinning a service-oriented model for (sensing) infrastructure out from the platform that is meant to exploit it, sandboxing and access to hardware resources need to be crafted explicitly for the platform itself, instead of delegating such duties to an IaaS-level framework.

With regard to the deployment of sensing applications and tasks, the above frameworks allow for rapid installation and execution on all contributing devices indiscriminately. Additionally, Medusa provides specific rewards and incentives to stimulate user participation while it allows smartphone owners to specify limits on usage of system components. However, most of these frameworks do not provide any advanced mechanisms for targeted selection of contributing devices, despite that such a process could significantly increase efficiency in the exploitation of available resources, as well as minimize impact on existing MCS-dependent

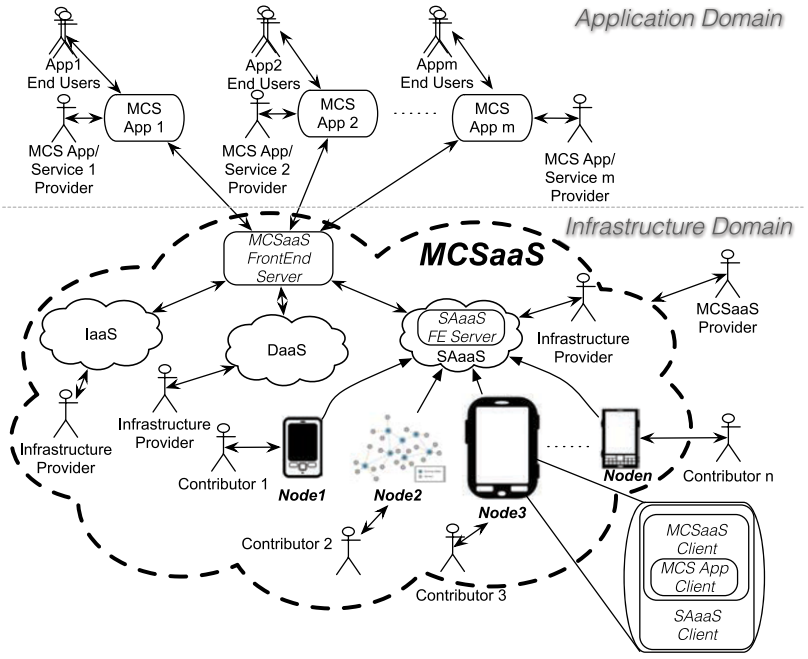


Fig. 2. MCSaaS scenario.

services running on the same devices. This is a consequence of the lack of infrastructure ready to be exploited in a controlled way for MCS, as resources are still to be enrolled with ad-hoc mechanisms as provided by the aforementioned platforms.

In closing, it is notable that none of the above frameworks tackles support to contribution churn as a platform-provided feature, as a way to enable on-demand expansion or shrinkage of the user base of MCS apps, according to the needs of application developers and providers.

3. The MCSaaS paradigm

3.1. The MCSaaS vision

The MCSaaS framework attempts to address the basic limitations of existing MCS applications, such as explicit and time-consuming developer efforts in the engagement of resources, while instead supporting contributor recruitment by MCS ASPs. Specifically, in MCSaaS these issues are addressed by making infrastructure available on demand. This way, MCS application/service providers can immediately deploy and run their applications and services on promptly available resources, ready to use once configured or customized for deploying the MCS application. Another important benefit of this approach lies in having real-time requirements fulfilled mostly for free, by design due to the paradigm shift, since QoS requirements can be satisfied by providing a certain guaranteed number of devices in face of loss.

From a high level point of view, this approach can be implemented by functionally and administratively splitting the MCS application/service deployment into two domains: (i) the *infrastructure* domain, which includes (embedded) sensing devices, providing services for resource management (brokering, interoperability, etc.) and facilities for customizing and deploying the MCS application; and (ii) the *application* domain, hosting the frontend and backend (analytics) services for the (filtered and pre-processed) data provided by the infrastructure modules, exploiting infrastructure/low level *application deployment* and *data/node management* facilities to implement the MCS application or service.

This strongly impacts on the MCS paradigm, significantly changing the scenario by introducing new stakeholders, and enabling further avenues for exploitation, not only in terms of research but also from a business perspective, such as an open market of MCS(aaS) sensing resources and services. More specifically, as can be seen in Fig. 2, different application service providers (MCS ASPs) may leverage the facilities an MCSaaS platform provider has on offer, including traditional enterprise-level infrastructure providers such as the incumbent players for both processing and storage resources, i.e., Infrastructure as a Service (IaaS) and Data as a Service (DaaS), respectively. Under the sensing Cloud a few kinds of infrastructure contributors are depicted: as we are talking about SAaaS here, the owners/admins are contributors sharing resources under their control, such as mobiles, PDAs or even Wireless Sensor Networks (WSNs).

The differences between the "traditional" MCS scenario and the proposed MCSaaS one are clear by comparing Figs. 1 and 2, and have been summarized in Table 1, where the main MCSaaS actors with their duties, as well as the pros and cons of their roles, are described. More specifically, within the "plain" MCS domain (Fig. 1), a potential contributor directly interacts with an MCS ASP to be engaged in an MCS activity. According to the MCSaaS scenario in Fig. 2, contributors are node owners/administrators that are sharing resources under their control via registration to a sensing Infrastructure Provider (SaaS), possibly retained by means of suitable incentives. The contribution is therefore managed at a lower level, thus lending a degree of freedom to resource sharing (multiple MCS app contributions, contribution profiles, credits, money, etc.) but this requires to delegate resource control to the infrastructure provider. The MCSaaS scenario is thus enriched by new stakeholders, such as the Infrastructure Providers (InP) and the MCSaaS Provider (MCSaaS-P) in between the MCS ASP and the Contributors. This brings about several benefits and is advantageous in particular for MCS ASPs, who can rapidly develop and deploy their apps onto MCSaaS-enabled infrastructure.

A service-oriented provisioning model is the best way for the MCSaaS-P to provide the required resources to MCS ASP, enabling customization facilities while ensuring the required service levels for provisioning. The MCSaaS-P can provide support to single or multiple MCS applications/services to be delivered transparently

Table 1
Actors involved in the MCSaaS scenario.

Actor	Description	
	Pros	Cons
Contributor	*Volunteering contributor of sensing resources, stimulated by appropriate incentives.* – Possibility to earn credits, rewards. – Free or remunerated. – Multiple MCS application contributions through a single subscription (delegation pros). – Possibility to be both user and contributor. – Privacy and security due to two layers of isolation.	– The node contribution is initially managed by a third party broker (delegation cons).
InP	*A sensing infrastructure provider enrolls and manages contributors according to specific service level agreement.* - Enlarge the business customer portfolio (due to mashups).	- Third party brokering and monitoring.
MCSaaS-P	*The MCSaaS-P provides resources mashup and brokerage services to MCS ASPs, along with customization service for the engaged resources and MCS application deployment service.* – New business opportunities. – Increased resource utilization and throughput. – To reach a wide audience. – Big data-analytics. – Involving sensor networks. – Actuation.	– Resource management. – Duties on security, privacy and SLA to both sides.
MCS ASP	*Application provider that delivers a single or multiple MCS applications/services to MCS End Users. The MCSaaS ASP deploys the MCS application(s) utilizing the MCSaaS framework.* – Wider application domains involving mobile and/or static (SN) sensors and actuators. – No problems of enrollment and management of sensing resources. – QoS-guaranteed resource provisioning. – Real-time applications suitability. – Increased application reliability, availability and performance. – Capillarity, worldwide coverage, # of contributors. – Heterogeneous resources (computing, storage, sensing) provided. – Facilities for the application deployment (configuration, customization, setup, analytics tools). – Customizability of resources (pre-processing, filtering, client-side functionalities, reduced overhead, bandwidth-local processing trade-off). – Abstracted/homogeneous access (APIs) where resources are what is customized to meet the needed abstraction (SAaaS-unique). – Resources handling capabilities due to the device/resource-centric approach vs the data-centric one, enabling enhanced features and new applications. – New applications involving actuation resources.	– May be charged.
MCS EU	*The consumer of the MCS services provided by the MCS ASP.* – High performance, low delays. – The role of an MCS EU may naturally coincide with the role of Contributor.	- May be charged.

to MCS End Users (EU), as in the traditional MCS scenario. In order for the MCSaaS-P to provide the requested sensing resources to the MCS ASP, the two parties have to negotiate the set of required resources and then, upon agreement, the ASP deploys the MCS app to the corresponding set of registered contributing nodes provided by the InP through the MCSaaS platform. Details on this process are reported in Section 6.

3.2. The MCSaaS stack

The main idea we propose in this work is therefore to adopt a Cloud and service-oriented approach for the on-demand provisioning of MCS resources and services. This way, the SAaaS sensing Cloud becomes a pillar of the MCSaaS infrastructure. Furthermore, a higher platform-like layer to provide services for the resource management (mashup, brokering, interoperation, etc.) as well as for deploying and customizing the app on top of such infrastructure is mandatory. Sensing and actuation resources are involved in the Cloud not as simple endpoints, as in current mobile Cloud trends, but rather in the same way as computing, storage, and network resources usually are in more traditional Cloud stacks: abstracted, virtualized and grouped in Clouds, to unlock new value-added services by mixing the potential of the Cloud with that of the IoT.

Our goal is therefore to provide a conceptual framework, and correspondingly a software stack, able to deal with such issues, while aiming at the MCSaaS vision as the whole. To this purpose, we adopt a multi-tiered approach as depicted in Fig. 3, where a layered scheme comprising the node and the infrastructure Clouds below the platform (PaaS) and the application-Software as a Service (SaaS) on top, is proposed.

At a lower level of the MCSaaS stack we have contributors sharing their nodes with infrastructure providers that enroll them to provide (virtual) sensing and actuation resources as a service (SAaaS). Optionally also computing (IaaS) and storage (DaaS) providers could be included at this level. On top of the basic infrastructure mechanisms, services mainly related to the specific configuration, customization, and management of virtual resources for MCS applications are provided by the MCSaaS Cloud platform. Facilities to expose resources provided by different categories (i.e., IaaS, DaaS, SAaaS) of infrastructure providers may be implemented by the MCSaaS platform. At a higher level, an MCS application employs such infrastructure and platform facilities to eventually enable and provide SaaS services. However, an MCS ASP may just deploy an application, collecting and/or displaying data, without necessarily building up a Web service out of it, or otherwise becoming an (MCS-powered) Software as a Service provider.

4. The Infrastructure

As seen in the previous section, the MCSaaS scenario may involve third parties to provide IaaS/DaaS resources where needed for developers of MCS applications, but here we focus on the

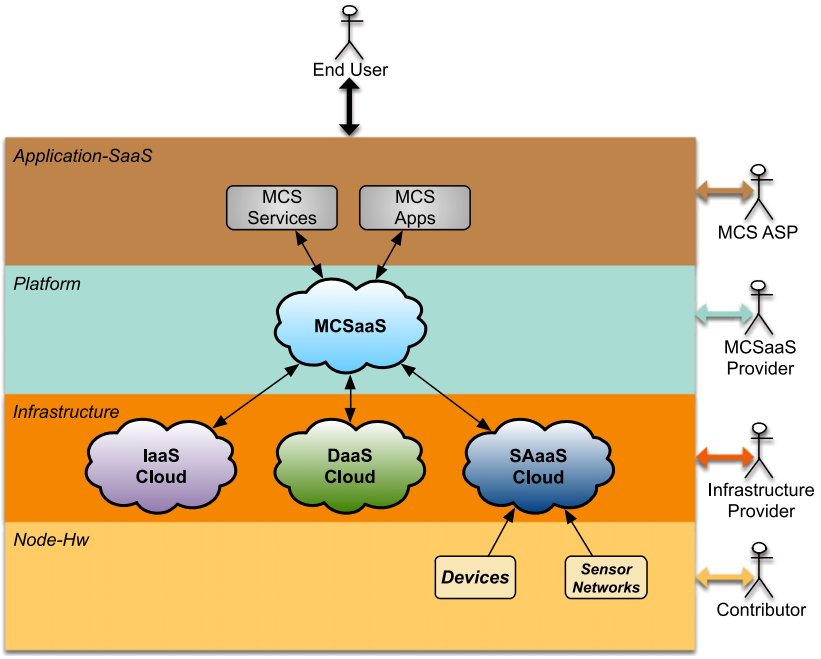


Fig. 3. MCSaaS stack.

main building block of any kind of MCS-centered facility: (mobile) sensing infrastructure, under the guise of service-oriented Cloud-enabled resources, a so-called Sensing and Actuation as a Service (SAaaS).

SAaaS is a paradigm aimed at developing a sensing infrastructure based on sensors and actuators from both mobile devices and SNs, providing virtual sensing and actuation resources in a Cloud-like fashion. More specifically, it delivers the basic mechanisms and tools for enabling a Cloud of sensors and actuators, which have to be suitably extended and customized by providers to implement enhanced services and provisioning models. To this end, the main issues to be addressed include: abstraction of sensing and actuation resources, virtualization, customization, monitoring, SLA and QoS management, subscription, churn and policy management, enrollment, indexing and discovery, security and fault tolerance.

This fosters the design of a software stack that implements the following main functionalities: (i) involvement of SNs, smartphones or other devices endowed with sensors and/or actuators, and their enablement for interoperation in a Cloud environment; (ii) distributed mechanisms and tools for self-management, configuration and adaptation of nodes; (iii) functions and interfaces for enabling and managing contributing resources.

To implement such ambitious idea, i.e., a Cloud of sensors based on the SAaaS paradigm, in [6] we introduced the whole stack and a high-level blueprint of the architectural modules, the three main components of which are shown in Fig. 4, bottom-up: Hypervisor, Autonomic Enforcer and VolunteerCloud Manager.

The SAaaS stack and modules span to the two lower layers of Fig. 3: from the Cloud Infrastructure one, providing support to the MCSaaS PaaS and MCS SaaS software applications and services, to the node layer, covering *edge* devices. Through the SAaaS stack, any device, either mobile or static, may be engaged in crowdsensing activities, as well as any sensor network, regardless of the software environment and operating system. This way, the term SAaaS *node* is used in the following to indicate a smart device equipped with sensors, such as a smartphone, or a frontend to a possibly large number of smaller sensing devices (i.e., motes), such as the gateway of an SN.

Furthermore, since the SAaaS implements a *device-centric* approach, providing actual sensing and actuation resources, even

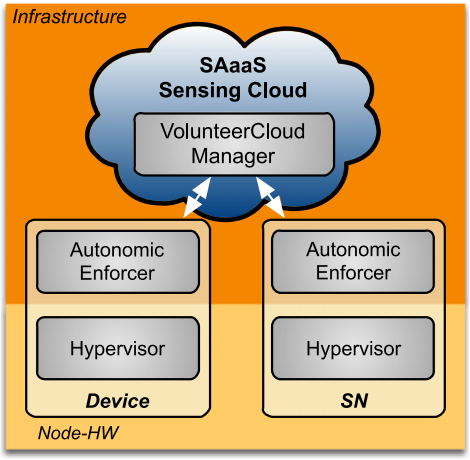


Fig. 4. SAaaS reference architecture: modules.

if multiplexed and/or virtualized, it allows for customization and configuration capabilities typically unexposed higher up to MCS ASPs, that may in turn explore previously neglected application domains.

The lowest block, the Hypervisor, operates at the level of a single node, where it abstracts the available sensors. The node can be a standalone resource-rich device, such as a smartphone, or it can be an embedded system which belongs to a network (such as a WSN). The main duties of the Hypervisor are: communications and networking virtualization primitives, abstract description of devices and capabilities according to the relevant information model, virtualization of sensing resources, customization, isolation, semantic labeling.

The Cloud modules, under the guise of an Autonomic Enforcer and a VolunteerCloud Manager, deal with issues related to the interaction among nodes. The former is responsible of the node-internal enforcement of Cloud policies, local vs. global policy tie-breaking, cooperation on overlay instantiation, enrollment initiation and subscription(s) bookkeeping. Whenever suitable it operates according to autonomic principles. The latter is instead in charge of the following functionalities: exposing the Cloud of

sensors via Web Service interfaces, indexing of subscribers and contributed resources, monitoring of Service Level Agreements (SLAs) and Quality of Service (QoS) metrics. These layers thus form a coupled, two-level Cloud stack, where many mechanisms are split in both modules, dealing with node-wise actions and self-organizing properties in the lower one, and centrally managed Cloud-wise methods in the upper one.

Indeed, taking as an example device enrollment, the lower Cloud module is in charge of node-side initiation of the enrollment process, including one-time interaction with the contributor, e.g., for excluding or limiting access to certain device-hosted resources, by pushing the description of enumerated resources to the Cloud, as well as of bookkeeping of any Cloud instance the node has successfully subscribed to. The centralized module is instead in charge of Cloud-side acceptance or rejection of the enrollment request as well as indexing of resources (and the corresponding nodes) to provide querying capabilities to the end user of the Cloud.

At this level, having communication among the Cloud modules layered on top of ubiquitously available protocols and services for IoT and M2M such as WebSockets [28] means getting access to lots of available gadgets and personal devices that would otherwise go untapped, unless major revisions in terms of their firmware, and communication stacks in particular, get planned. This choice directly reverberates on the widening possibilities a MCS app developer may experience as a result of the potential expansion of the pool of resources.

5. The platform: MCSaaS module

For the design of the MCSaaS stack, the need comes up to define what an MCSaaS platform should offer. PaaS is an intermediate service model for Cloud computing, between IaaS and SaaS, where the consumer creates software using tools, libraries, etc., available from the provider, also controlling software deployment and configuration settings. The PaaS provider usually hosts lots of ancillary facilities to the main infrastructure under consideration (in our case SAaaS mainly), e.g., networks, servers, storage, sensors.

A platform like MCSaaS must cater for domain-specific APIs for MCS, such as pre-filtering and pre-processing mechanisms to be leveraged both at the node endpoint (e.g., the mobile) and at the Cloud, while considering the trade-off between communication overhead and local computation. Other APIs would include general-purpose analytics frameworks, in this case to be leveraged only at IaaS/DaaS level. Issues related to federation, inter- or multi-Cloud setups, privacy, security and trust, which would ideally fall under the scope of MCSaaS, have not been investigated for this specific effort. Thus application hosting and a deployment environment are the main focus in this context. Moreover a PaaS typically also includes facilities for application design, application development and testing as well as typical services for developers and integrators, such as team collaboration, Web service integration, database-driven persistence, state management, application versioning, application instrumentation / profiling and facilities for community nurturing.

All aforementioned functionalities are synthesized in the MCSaaS module implementing a PaaS service at the platform layer (see Fig. 5). The *Infrastructure Interface* provides the means to interact with the underlying sensing Clouds. Standard interfaces such as OCCI [29] and/or specific techniques following the multi-Cloud approach can be adopted. On top of the Infrastructure Interface, four MCSaaS core sub-modules are defined, namely Cloud Broker, Orchestrator, Customization Service and Deployment Manager.

An incoming MCSaaS request is forwarded by the *Platform Interface* to these sub-modules. Such interface should be RESTful and expose as entities useful abstractions, e.g., application bundle

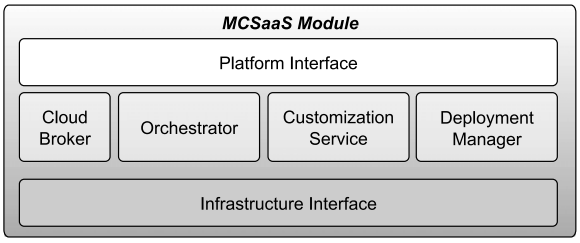


Fig. 5. MCSaaS module.

objects. This is initially managed by the *Orchestrator*, which identifies the resources required by the MCS application extracting requirements and dependencies from the request encoding the high level workflow of the application. This kind of service may conform to available standards, e.g., TOSCA [30], in terms of exposed functionalities and relevant APIs. It therefore iteratively interacts with the *Cloud Broker* to reserve resources according to the aforementioned requirements provided by the MCS ASP, the Broker in turn planning in advance the engagement of sensing devices as needed through SAaaS InPs. Following a successful negotiation and the allocation of required resources to the MCS application the application setup phase is launched by both the Customization Service and the Deployment Manager. The *Customization Service* mainly focuses on customizing the resources and (virtual) devices, setting specific low-level parameters such as duty cycle, sampling frequency and scale range for sensing resources, as well as high level configurations of the software environment hosting the injected code of the MCS application. This is achieved by translating high-level directives in terms of uniform low-level primitives, still expressed in a generic form, as those are ultimately relayed to the SAaaS for further processing. The *Deployment Manager* instead is aimed at deploying the required modules on the available resources, adapting them to the hosting environment. Pre-configured packages or bundles may be provided, where the payload may contain a whole application environment or parts of it, or even extra tools that implement advanced features such as analytics, interfaces, domain-related plugins and add-ons. As the latter modules are here the main focus for the initial implementation, in Section 6 the workflows involved are described and in Section 7 more details are given in terms of the corresponding software design.

6. Setup and deployment of MCS applications

Within the proposed paradigm, the actions and interactions of the main stakeholders (depicted in Fig. 2) through the blocks and the modules identified above, are of high practical importance. Thus, the main activities related to MCS application setup, deployment and management into the MCSaaS framework, are described through the following Activity Diagrams (AD). More specifically, two main perspectives associated with the MCSaaS-P and the MCS ASP are taken into account in the description of these main activities, which include (i) setting up an MCSaaS platform (Section 6.1), followed by (ii) the configuration and deployment of a specific MCS application on it (Section 6.2), respectively.

6.1. MCSaaS platform setup

With regard to enablement of PaaS over potentially exploitable infrastructure (SAaaS), we first need to describe a bootstrap scenario, where mandatory MCSaaS components are pushed to the mobiles in a PaaS-agnostic way. In more detail, the following kind of interaction is envisioned: the MCSaaS provider needs to choose the appropriate infrastructure, given a set of available InPs, to offer MCS as a service.

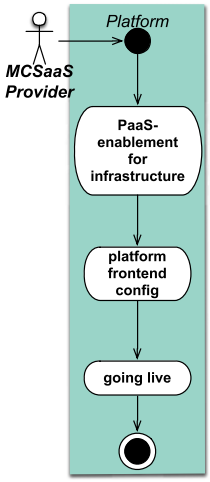


Fig. 6. Initial platform setup.

Afterwards a phase of bootstrap ensues, where the MCSaaS-P must enable every mobile, booked through the SAaaS InP, for easy MCSaaS-assisted deployment of MCS applications. For this purpose SAaaS-provided services, powered by the Hypervisor, are employed to deploy the node-side MCSaaS module.

The MCSaaS module enables contributors to choose their level of involvement and resource sharing for MCS apps, i.e., not only sensing resources as in pure SAaaS scenarios, but also CPU time, memory and/or on-board storage. Moreover, this module is in charge of duties related to MCS app deployment, i.e., activating endpoints for, and then cooperating with, the Deployment Manager in Fig. 5, also enforcing the aforementioned choices in terms of local computing and storage resources when receiving and deploying the next MCS app.

The aforementioned approach, apart from avoiding layering (IaaS/PaaS) violations, also enables us to implement the mobile-side SAaaS components to cope just with sensing infrastructure concerns and SAaaS scenarios. This focus then carries over also in terms of sandboxing and other security-related mechanisms, e.g. dealing with user-mandated restrictions, only with regard to SAaaS-relevant sensing resource usage and interactions.

In the same fashion, any node-side operations dictated by the Customization Service, would just be dealt with by the bootstrapped environment, thus leading to the development of certain security-related mechanisms, such as checking permissions of MCS apps in terms of, e.g., access to any kind of user data, to be implemented only at this level, i.e., in the MCSaaS bootstrapped component itself.

Fig. 6 depicts MCSaaS-P activities related to the platform setup, starting with a macro-step including all actions involved in *PaaS-enablement for infrastructure*, such as identifying and selecting potential InPs, exposing relevant APIs and service endpoints. Afterwards a phase of *platform frontend configuration* follows: this entails the setup of a Software as a Service instance, i.e., a Rapid Application Development (RAD) environment available to application developers. As soon as the frontend is ready, the MCSaaS-P can "go live" and start servicing customers, e.g. MCS ASPs.

6.2. MCS application configuration and deployment

The MCS application configuration/deployment is performed by the MCS ASP using the services provided by an MCSaaS-P and is broken down into the (i) application negotiation and platform bootstrap stages and (ii) the actual app deployment, detailed in Sections 6.2.1 and 6.2.2 respectively.

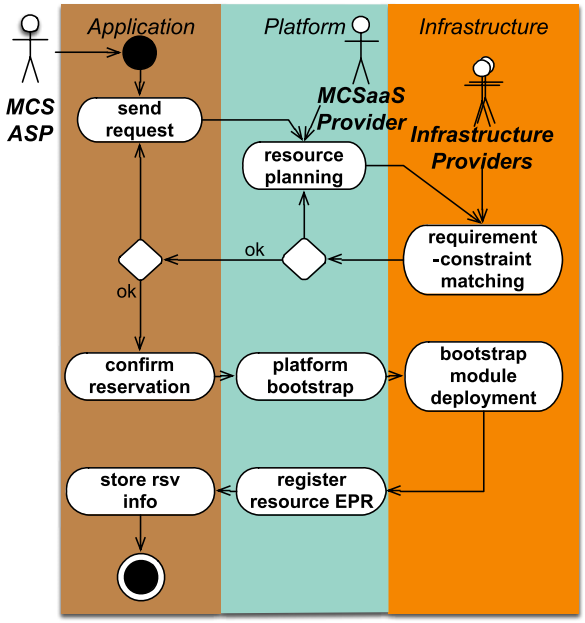


Fig. 7. App negotiation and platform bootstrap.

6.2.1. Negotiation and bootstrap

Fig. 7 describes the (PaaS-mediated) app negotiation and platform bootstrap stages. The former entails identifying the required building blocks for the app, in terms of IaaS/DaaS/SAaaS resources, with a phase where the developer (or MCS ASP) pushes the corresponding plan to the platform frontend (*resource planning*). It is followed by a negotiation of the requirements to be matched, coordinated by the Cloud Broker, possibly with iterations involving the MCS-ASP still into the planning stages (*requirement-constraint matching*). Upon agreement and following the MCS ASP confirmation (*confirm reservation*), the platform modules have to be deployed into the provided sensing resources, through the interaction between the MCSaaS-P (*platform bootstrap*) and the InP (*bootstrap module deployment*). Thus, the end-point references (EPR) of the reserved sensing resources are fed back to the MCSaaS-P and stored (*register resource EPR*) and the reservation data forwarded to the MCS ASP (*store rsv info*).

6.2.2. Deployment

Field deployment of an MCS app, as detailed in Fig. 8, starts with the MCS ASP choosing among available APIs (*choice of API*), for basic as well as advanced operations (e.g. pre-filtering, analytics). This is followed by the platform's *artifact contextualization* of the MCS application and the configuration of the infrastructure sensing resources on the specific application domain (*app domain configuration*), translating at the same time customer-driven constraints on resources and APIs by wiring up infrastructure endpoints accordingly (*service endpoint wiring*).

Then, an initial set of requirements is submitted by the application (*requirement push*), which is the "recipe" (including high-level code) obtained during the bootstrap phases as discussed in Section 6.2.1. This step kickstarts the *artifact deployment* (binaries, configurations, system images, etc.) to VMs, storage objects and, in the case of SAaaS, contributor-owned mobiles. Behind the scenes the deployment would be preceded by translation of recipes in artifacts, e.g. compilation/packaging, still up to the Deployment Manager. Activation of the endpoints, and subsequent mapping of the wiring (as shown in the corresponding AD of Fig. 8) over allocated resources (*orchestration*), are up to the Orchestrator. For the operations of the Orchestrator to be

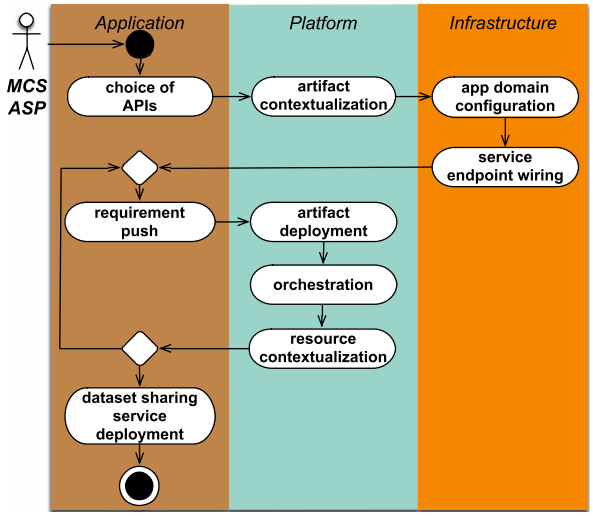


Fig. 8. MCS app deployment.

effective, or even just feasible under most circumstances, platform-initiated activations and (re)wirings are required, thus leading to a need for an always-on (anytime) push-to-client messaging channel for each registered device, powered by environment-agnostic bi-directional asynchronous exchange primitives.

Afterwards, a phase of *resource contextualization*, by means of the Customization Service, is called for, either in terms of configuring or even customizing the underlying resources, e.g., by iteratively involving the SAaaS services, where relevant. Mapping and, especially, customization could possibly lead to a mismatch between initial requirements and actual setup, thus requiring eventual iterations with MCS ASP to reach a convergence or at least a satisfactory trade-off before stopping the matchmaking process. Furthermore, the MCS ASP can start exposing a portal or useful Web services to share and visualize full datasets, or any compact representation thereof in the *dataset sharing service deployment*. These datasets may therefore be provided to third parties (also as OpenData) to let them develop applications centered around such datastores.

7. The MCSaaS design

In the following we provide a description and some details regarding the MCSaaS stack implementation.

With regard to the node-side runtime, there are the Sensing APIs and the environment-provided notification subsystem which are of particular relevance to our design. Whilst the former is opaque in terms of the MCSaaS, as its Customization Service has to relay sensor reconfiguration and tuning requests to the SAaaS device-side subsystem, the latter is useful at both Cloud layers, to minimally involve platform-provided Cloud-based mechanisms for push-based communication to devices, avoiding to tackle corner cases in terms of reachability, and tracking of networking conditions in general.

The server-side logic, available in the Orchestrator and the Cloud Broker, has been developed in Java and Python, resorting to servlets [31] in terms of the user-facing interaction model and RESTful endpoints as Cloud-private interfaces to the mechanisms implemented in Python.

Apache Tomcat [32] is used as the Java servlet container. For example, Google notification service, Google Cloud Messaging (GCM) [33], is used for exchanging (MCSaaS-bootstrapping) asynchronous messages in Android-powered mobiles, as needed to support Cloud-initiated primitives and runtime customizable mechanisms, as well as for on-demand activation of a WebSocket-based subsystem. This preference towards a (partially) custom communication bus lies in the inherent limitations (and costs) borne by the use of GCM, which is simply not meant for big payloads (e.g., file transfers), but is specifically designed and marketed for push notifications and in our case employed also for signaling.

In terms of the main modules, the Broker keeps track of the reservations and transfers resource requests to the Orchestrator. The latter in the current implementation mainly deals with churn, which at MCSaaS level means reacting to fluctuations in the number of active contributors for any of the MCS apps hosted by the system. In particular, as soon as in a certain area such population falls below a threshold, according to parameters set by the MCS developer, the churn management routines running in the Orchestrator look up other platform-enabled nodes in the same area to push notifications through GCM, meant for triggering the nodes to retrieve the package, install and launch the corresponding payload, the MCS app. The whole MCSaaS population is continuously under tracking so any query is expected to be serviced in the order of fractions of a second. The client-side component of the Deployment Manager listens for incoming payloads, tagged as such by the server-side logic, to automate the installation of the app as soon as it is downloaded onto the device. This is to let the app be installed without any user intervention, albeit the process gets inevitably visible at times, as some windows would pop up anyway during installation stages, albeit for very short time intervals (e.g., less than a second for any window instance).

Fig. 9 depicts the layout of the infrastructure and platform-level framework subsystems, the *MobileCrowdSensing Open Stack* (MobiCSOS), focusing on communication between end users, e.g., MCS ASPs and other interested developers, and sensor-/actuator-hosting nodes. On the node side, the *MobiCSOS Edge*, acting either as SAaaS Client or MCSaaS Client according to the corresponding instance of execution, interacts with the OS resources and services, and in the case of SAaaS, with (virtualized) sensing and actuation resources through (mediated) access to the Sensing APIs. It is the point of contact with the Cloud infrastructure, allowing end users to manage node-hosted resources even if they are behind a NAT or a strict firewall. This is ensured by a GCM and WebSocket-based communication between the MobiCSOS Edge and its Cloud counterpart, namely the *MobiCSOS MPlatform service*. The MobiCSOS MPlatform service, acting both as VolunteerCloud Manager (SAaaS) and Broker (MCSaaS), is implemented as an OpenStack service, providing Cloud users with the ability to book and manage SAaaS or MCS-contributed resources remotely, respectively. This can happen both via a command-line based client, namely *MobiCSOS command line client*, and a Web browser though a set of servlet-wrapped RESTful APIs provided by the MobiCSOS MPlatform service.

7.1. Node-side components

With respect to network connectivity and presence/reachability, *WebSockets* [28] are our technology of choice, a standard HTTP-based protocol providing full-duplex TCP communication over a single HTTP-based persistent connection. One of the main advantages of WebSocket is that it is network agnostic, and clients only need outgoing traffic (over standard ports for the Web) to be allowed. This is of benefit for those environments which block any Internet connections unrelated to Web traffic using a firewall.

Fig. 10 shows the MobiCSOS architecture with more focus on the node side SAaaS components, whereas Fig. 11 shows the architecture with regard to the MCSaaS instance of node-side MobiCSOS components.

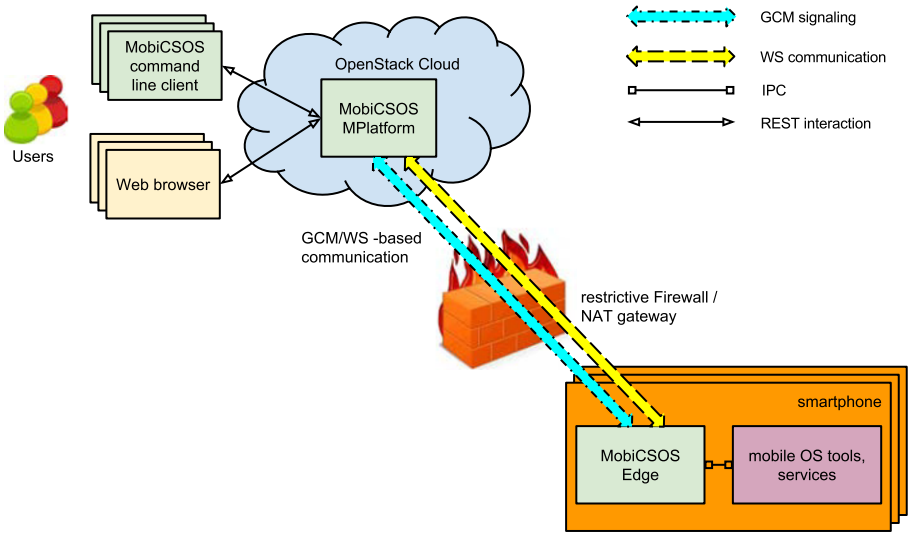


Fig. 9. MobiCSOS subsystems, for mobile-based nodes.

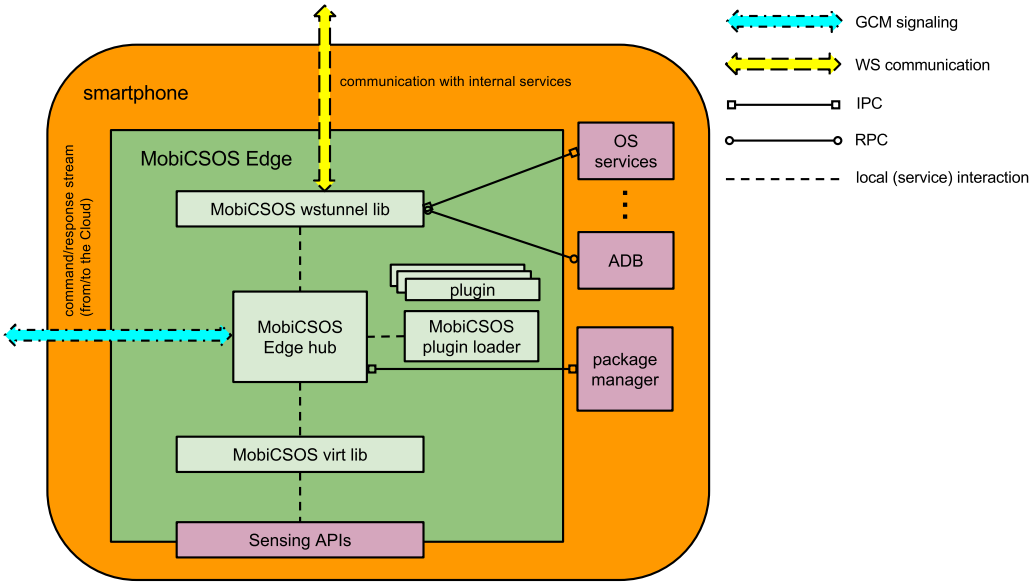


Fig. 10. MobiCSOS node-side SAaaS architecture.

Building upon the Sensing APIs abstraction at the lowest level, there is the need for the SAaaS node-side subsystem to mediate access by means of a set of virtualization-inspired libraries, denoted as *MobiCSOS virt libraries*. These provide the interface to read/write sensor/actuators as well as to set device parameters, thus are an actual implementation of the SAaaS Hypervisor. Such operations are thus placed at the right level of semantic abstraction, i.e., locking and releasing resources according to bookings and in a way that is dependent upon constraints deriving from the granularity of the APIs and that are specific to the sensing and actuating resources.

The *MobiCSOS Edge hub* represents the core of the board-side software architecture, thus can be considered as the centerpiece of the SAaaS Autonomic Enforcer, whereas a specialized instance of the same component is hosted in the MCSaaS client. This engine in both instances interacts with the Cloud through a WebSocket full-duplex channel and GCM-based messaging (see also Fig. 12), sending and receiving data to/from the Cloud and executing commands provided by the users via the Cloud, respectively. Such commands can be related to the communication with the node sensing and actuation resources (through the *MobiCSOS virt libraries*) and to the interactions with OS resources and system-level tools (e.g., filesystem, background services and activities, package manager). Communication with the Cloud is ensured by SDK-provided primitives for push-based messaging, in our case GCM under Android.

Moreover, a set of WebSocket libraries (*MobiCSOS wstunnel libraries*) enables the engine to act as a WebSocket-based (reverse) tunneling server, connecting to a specific WebSocket server running in the Cloud. This allows internal services to be directly accessed by external users through the tunnel, whose incoming traffic is automatically forwarded to the internal background service under consideration. Outgoing traffic is redirected to the tunnel and eventually reaches the end user that connects to the WebSocket server running in the Cloud to interact with the node-hosted service.

The MCSaaS gets (selective) access to the (virtual) sensing resources by means of the *MobiCSOS sensing proxy*, as well as to mobile-hosted services and in particular the package manager, through the corresponding proxies as depicted in the diagram. All such proxies interact with the SAaaS instance of the *Edge hub*.

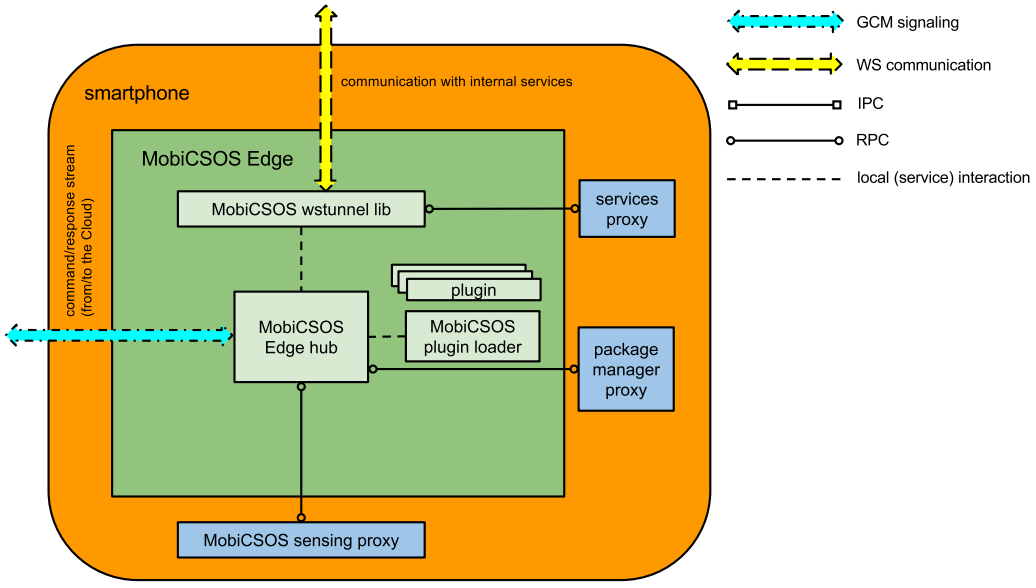


Fig. 11. MobiCSOS node-side MCSaaS architecture.

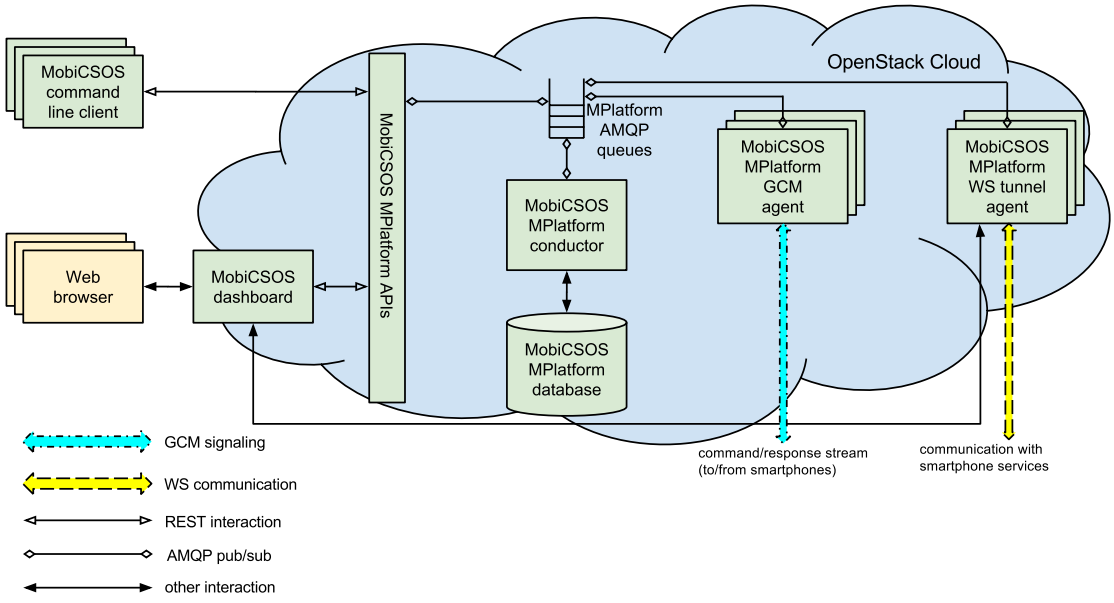


Fig. 12. MobiCSOS Cloud-side architecture.

The *Edge* subsystem also provides a plugin loader which, in the MCSaaS instance, implements the node-side mechanisms enabling the Customization Service. Custom plugins can be injected from the Cloud and run on top of the plugin loader to implement specific user-defined primitives, including (sandboxed) system-level interactions, such as, e.g., with a package manager and/or services automatically started at boot-time. Indeed, one of such plugins, the one interacting with the package manager, implements the node-side counterpart to the MCSaaS Deployment Manager.

7.2. Cloud-side architecture

As already mentioned, with respect to virtual infrastructure management, OpenStack [34] is the technology of reference. OpenStack is a centerpiece of infrastructure Cloud solutions for most commercial, in-house, and hybrid deployments, as well as a fully Open Source ecosystem of tools and frameworks upon which many research projects are founding their Cloud strategies. Hence our goal is to extend OpenStack for the management of sensing and actuation resources and to support a platform for MCS.

The MobiCSOS Cloud-side architecture (see Fig. 12) consists of an OpenStack service we called *MPlatform*. MPlatform is characterized by the standard architecture of an OpenStack service. The *MobiCSOS MPlatform conductor* represents the core of the service, managing the *MobiCSOS MPlatform database* that stores all the required information, e.g., node-unique identifiers, association with either SAaaS and/or MCSaaS users and tenants, board properties and hardware/software characteristics as well as dispatching remote procedure calls among other components. Indeed, it is the component where the Broker and Orchestrator logic is implemented, otherwise interacting with external ad-hoc services implementing mechanisms for customization and deployment, here comprising an instance of a continuous integration subsystem, i.e., Jenkins [35].

The *MobiCSOS MPlatform APIs* exposes a two-fold RESTful interface for the end users, one for SAaaS and the other for

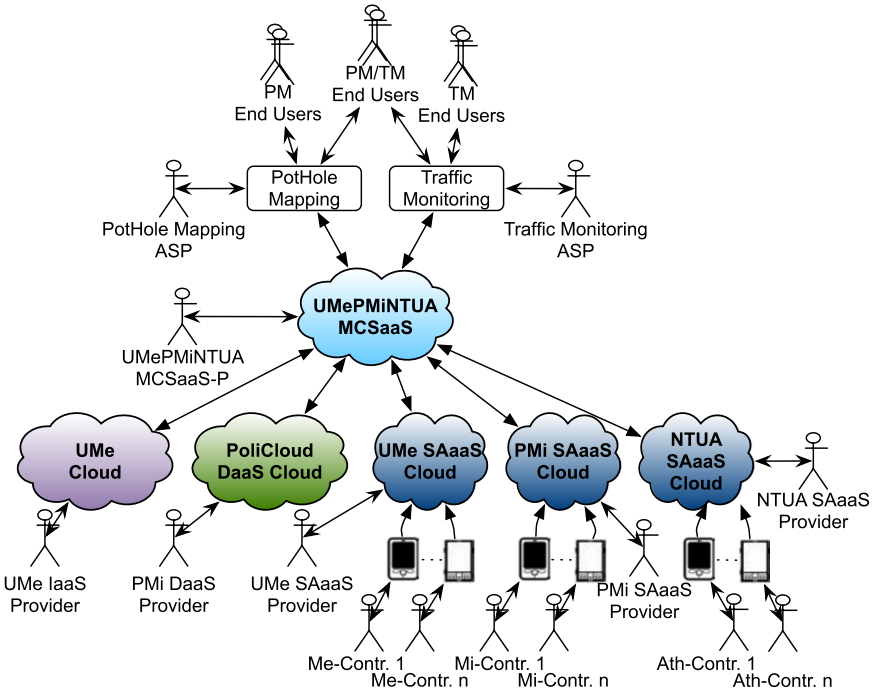


Fig. 13. MCSaaS-driven app deployment scenario.

MCSaaS, that may interact with the services both via a custom client (*MobiCSOS MPlatform command line client*) and via a Web browser. In fact, the OpenStack Horizon dashboard has been enhanced under the guise of a *MobiCSOS dashboard*, exposing all the functionalities provided by the MobiCSOS MPlatform service and other software components. In particular, the dashboard also deals with access to node-internal services, redirecting the user to the *MobiCSOS MPlatform WS tunnel agent*. This piece of software is a wrapper and a controller for the WebSocket server to which the nodes connect through the use of MobiCSOS wstunnel libraries.

Similarly, the *MobiCSOS MPlatform GCM agent* acts as a GCM bridge between other components and the nodes. Indeed the command stream, such as the one originating from the Customization Service, gets relayed by the GCM agent. It translates AMQP messages into GCM messages and vice versa. Following the standard OpenStack philosophy all the communication among the MPlatform components is performed over the network via AMQP queues. This allows the whole architecture to be as scalable as possible given that all the components can be deployed on different machines without affecting the service functionalities, as well as the fact that more than one *tunnel agent* together with more than one *GCM agent* may be instantiated, each of them dealing in that case with a subset of the enrolled mobiles. This way, redundancy and high availability may also be guaranteed.

With regard to discovery mechanisms, which are core to the enrollment stages for the MPlatform conductor to register the available resources, the design is based upon the AllJoyn [36] framework for IoT, a family of standards and reference implementations which comprises at its core a DBus-derived application protocol useful for messaging, advertisement and discovery of services, working via selected mechanisms on available transports, in our case over MPlatform-enabled WS tunnels.

8. MCS app case study

In this section we discuss examples of MCS applications deployed using the MCSaaS platform, highlighting the benefits of MCSaaS. To compare the proposed approach against the traditional MCS one, we consider two possible use cases in an IoT/Smart City scenario, involving two different MCS applications: (i) a multi-purpose community-focused app for pothole mapping and (ii) a traffic monitoring app serving the Messina, Milan and Athens urban areas (Fig. 13).

The UMe (University of Messina), PMi (Politecnico di Milano) and NTUA (National Technical University of Athens) SAaaS (InPs) are used in the deployment scenario, leveraging their private IaaS and DaaS providers if needed (such as the OpenStack-powered UMe and PMi PoliCloud [37]). However, any private or public company/institution (e.g., mobile telcos) could take up the role of the InP. Our SAaaS Clouds feature smartphone sensors from volunteering Contributors (roaming users). Smartphones are bound to be extremely useful in such scenarios, as they are equipped with several relevant sensors e.g., for positioning (GPS, GSM, WiFi) and motion detection (gyroscope), while supporting (always-on) Internet access. Roaming users become Contributors by discovering and (non-exclusively) subscribing to SAaaS Clouds.

The UMePMiNTUA MCSaaS-P is established according to the workflow depicted in Fig. 6. After the selection of the appropriate InPs (subscribed to services, signed SLAs, etc.), UMePMiNTUA discovers, collects and exposes generic (templatized) WS endpoints (URIs), typically for RESTful consumption, and corresponding APIs documentation within an HTML5-powered RAD IDE.

On top of this platform, the pothole mapping and traffic monitoring apps are provided by two different MCS ASPs. Contributors may be engaged for both pothole mapping and traffic monitoring MCS apps concurrently, a desired outcome and one of the main drivers behind this effort. The MCS End User (EU) (e.g., a taxi driver), uses the corresponding mobile (or Web-based) apps for retrieving the information of interest. Often an MCS EU also acts as a Contributor.

In the following, we initially discuss how to set up and deploy these two MCS apps (Sections 8.1 and 8.2) using the proposed MSCaaS platform. Then, in Section 8.3, we (i) report on the preliminary results of the MobiCSOS prototype in the context of the traffic monitoring use case and (ii) highlight the benefits of the MCSaaS approach against the conventional MCS one through an analytic model.

8.1. Pothole mapping

A pothole mapping MCS application automatically collects road condition information once it is started, without human intervention. It requires just an accelerometer and a positioning system at the core, to be sampled for detection of abrupt vertical displacements and geo-localization. Data validation including the removal of outliers and elimination of false positives is facilitated by the crowdsensing approach collecting, analyzing and clustering input coming from a diverse range of users.

In the current MCS app deployment scenario, the pothole app requires just a database and a server for the Web UI. Therefore planning the requirements in terms of IaaS/DaaS is quite straightforward. On the other hand, the high-level constraints for the sensing infrastructure to be transmitted/negotiated include (i) sensing potential (accelerometers and positioning at least), (ii) geographical area and (iii) device mobility (vehicular, e.g., bikes and cars). The app design stage requires to leverage a set of libraries and ready-made routines, a typical process for a RAD environment, for, e.g., mobile-side outliers detection, as well as IaaS APIs to choose from (OGC- or M2M-compliant REST calls), etc., following the PaaS approach.

Once the app is released, the pothole ASP has two main alternatives: directly enrolling contributors supporting the pothole mapping service in a given area of interest (traditional MCS) or asking an MCSaaS-P, e.g., UMePMiNTUA, to provide the required (vehicular) sensing resources. As discussed above, in the former case, user enrollment could be slow or even not successful, while in the other case the MCS ASP has immediate access to the sensing infrastructure at the cost of the provided service (MCSaaS). Combining the two approaches in a kind of "cloudbursting" fashion, the pothole ASP may resort on-demand to third party sensing resources provided by the UMePMiNTUA MCSaaS-P along the "owned" ones directly engaged by the MCS, when required, e.g. in the case accuracy in the mapping is required within a given time constraint.

The pothole ASP has to negotiate with the UMePMiNTUA MCSaaS-P for the resources, required by the pothole mapping app that needs to be deployed in the MCSaaS platform/infrastructure. In the case of successful negotiation, the sensing resources provided by the UMePMiNTUA MCSaaS-P have to be set up and configured to receive the pothole app code as shown in Fig. 7.

The pothole app deployment begins as soon as the developer lets the output (*recipe*) of the design stage be consumed by the Deployment Manager, in this case just dealing with a limited subset of binaries (e.g., compatible with Android-powered mobiles), with regard to sensing resources (Fig. 8). Finally, at instantiation time endpoints that are provided/consumed by the application get enabled and the (abstract) wiring in the recipe gets mapped to the infrastructure by the Orchestrator.

8.2. Traffic monitoring

Another application of the MCS paradigm may be traffic monitoring, which is a useful instance of service based on mobile contributors, still related to drivers also as a potential pool of consumers and/or producers, and in general in the same domain. In terms of MCS, while the sensing activities are similar to pothole mapping, e.g., still mainly enabled by positioning subsystems and accelerometers, the requirements diverge remarkably.

As overall traffic and specific metrics, such as current cruising speed and the rate of slowdowns, to name a few, need to be fed and updated in *real-time* for the service to be genuinely useful, it follows that there are constraints such as, e.g., the minimum number of contributors per area on average, to ensure accuracy on traffic monitoring sampling. Most importantly the platform is meant for automatic deployment of multiple MCS apps if needed, ready to be executed side by side, so a scenario where both the pothole mapping application and the traffic monitoring ones are concurrently working in background is absolutely feasible and contemplated, while still addressing the specifics of the requirements of each application, e.g., real-time constraints on the availability of actively contributing resources in the case of traffic monitoring only.

The latter is really a use case for the orchestration capabilities of the platform, as exemplified by the management of node churn, which needs to be addressed on a continuous basis, to provide the expected quality of service, or at least degrading gracefully by informing the developer about the number of currently (or recently) involved contributors per area, in other words, under a simplified scheme, the confidence interval with respect to displayed metrics.

This way, resorting to the MCSaaS provider UMePMiNTUA is maybe the only effective solution to ensure a predefined level of accuracy in sampling for the traffic monitoring service. Also in this case a hybrid MCS-MCSaaS solution may be effective, exploiting the (vehicular) sensing resources provided by the UMePMiNTUA MCSaaS-P to complement and enrich the information gathered from the contributing ones engaged by the traffic monitoring ASP through a traditional MCS enrollment process. On the other hand, the enrollment of sensing resources from UMePMiNTUA follows the negotiation and deployment/configuration phases described in Figs. 7 and 8.

8.3. Testing and evaluation

To demonstrate the effectiveness of the MCSaaS model against the traditional MCS one, we evaluate (i) a preliminary implementation of the MobiCSOS stack for the deployment of the traffic monitoring app and (ii) the corresponding GSPN models.

8.3.1. MCS application deployment: A proof of concept

The traffic monitoring application, denoted hereafter as MCS-Traffic app, falls under the category of *infrastructure* MCS apps [1]. The implementation of the MCS-Traffic application follows the same principles as similar community-based GPS-enabled applications for navigation (e.g., Waze [38]), using contributor's location and travel times via GPS to provide real-time traffic updates, as exemplified in Section 8.2. To show the benefits of using the proposed paradigm for the MCS ASP, as opposed to current MCS application deployment practices, an illustrative example related to the MCS-Traffic app is presented, specifically focusing on issues such as volunteer-based contribution and node churn.

Testing scenario. The MCS-Traffic app requires real-time information (via GPS tracking) to provide updates on (expected) travel times for an arterial road (service area). Specifically, the minimum number of actively probing vehicles (contributors) is crucial for establishing the reliability of estimations in terms of link speed [39]. A link is a section of road spanning a continuous segment with no intersections, while the link speed refers to the distance traveled by a vehicle in a unit of time.

We consider the following testing scenario: vehicles enter the service area according to a Poisson process and travel times are normally distributed [40]. We assume that data from at least 10 probes are required with a sampling period of 700s to establish the reliability of the estimate on link speed [39]. This way, the traffic monitoring app is able to correctly estimate and predict actual traffic in the area of interest.

In our experiments we assume to have an average population of 150 vehicles in the service area. Furthermore, we assume that just a part of them are also SAaaS contributors (about 8%), and

similarly that just about 5% of vehicles are MCS contributors, directly engaged by the ASP. These values are arbitrary, however, since a conventional MCS application needs to recruit, engage and retain participants, we consider such assumptions reasonable, and maybe a bit conservative in the MCSaaS case.

Testing environment. The prototype implementation of MobiCSOS at MCSaaS and SaaS levels, described in Section 7, has been used to test the deployment and operation of the MCS-Traffic app. MCS EUs/contributors were emulated (running Android 4.2/API level 8 [41]), due to the lack of a substantial number of physical devices, as required by the testing scenario. We adopted Android, not only for its widespread availability and popularity, but also for some of the facilities the SDK and the runtime environment provide, e.g. the emulation subsystem (using Genymotion [42]) and the Debug Bridge (ADB) [43] for the management of emulated instances. Helper functions, using shell scripting, were employed for setting up the testing scenario (e.g., vehicle mobility, engagement, etc.) along with ADB for provisioning and control of the emulated instances.

Experimentation metrics. The two different scenarios discussed in Section 3 are examined in experiments based upon the MCS-Traffic app: (i) the one envisioned under the conventional MCS paradigm and (ii) the one enabled by the proposed MCSaaS paradigm.

We compare these two scenarios based on the average number of vehicles within the service area as a function of time, serving as active probes (*Number of Contributors*). For this purpose, we enumerate the (i) MCS contributors (MCS-C), involved in the conventional MCS scenario; and (iii) MCSaaS contributors (MCSaaS-C), contributing to the traffic monitoring app through the whole MobiCSOS stack. MCSaaS contributors are a subset of users engaged by the SaaS provider (InP) that are denoted as SAaaS contributors (SAaaS-C).

In addition, we quantify the effect of *Resource Churn*, by measuring on average the time required by the MobiCSOS framework to replace MCSaaS contributors that left the service area, restoring the MCSaaS pool to the number required by the MCS-Traffic app.

Results and comparison. In the MCSaaS scenario, it is up to the SAaaS provider (InP) to ensure a consistent high number of contributors engaged to a MCSaaS-P. Then it is up to the MCSaaS-P in its turn to reserve at least the minimum required amount of MCS contributors, according to the application requirements (in this case the MCS-Traffic app).

Number of Contributors: For the set of sampling periods, we estimated on average 5.8 contributors for the conventional MCS scenario and 9.2 contributors for the MCSaaS one, excluding a warm-up period of 500 s. Comparing the results obtained in the case of the MCSaaS scenario to the conventional one, we can argue that the MCSaaS solution succeeds in engaging contributors within the service area, as it approximately meets the constraint posed by the MCS-Traffic app in terms of the number of active probes (10 probes are required).

The effectiveness of the proposed approach in addressing the requirements of the application, is also visible in Fig. 14 that is a snapshot of the emulation within a sampling period, just reporting on a single testing trace. Following the 20th time unit there are two, almost simultaneous, SAaaS contributor departures that affect the MCSaaS provisioning service. However the MobiCSOS framework is able to promptly react, restoring the MCSaaS pool to 10 contributors/probes as requested by the MCS-Traffic app.

Resource Churn: Through the experiments we evaluated a lag of 6 time units (30 s) on average, due to the resource churn functionality, for the MCSaaS system to actually replace contributors (MCSaaS-C) that left the system. This lag is broken down into four intervals:

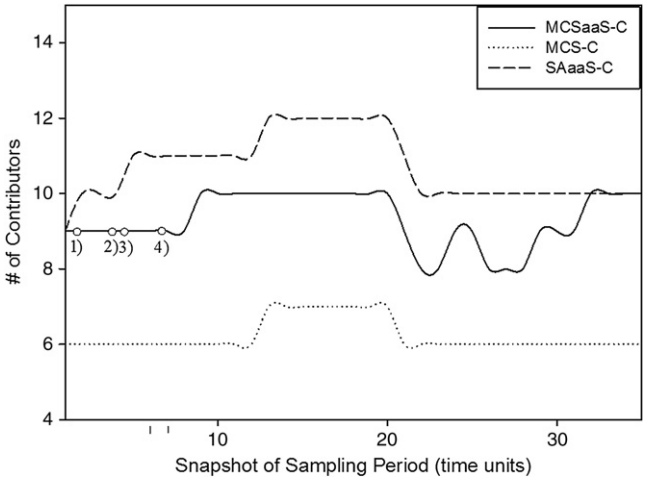


Fig. 14. Basic MCS and MCSaaS emulation scenario contribution sampling.

(1) the time needed for the SAaaS server to identify the arrival of a new contributor and initiate the MCSaaS client installation process;
(2) the MCSaaS client installation time;
(3) the time needed for the MCSaaS server to identify the arrival of a new contributor and initiate the MCS-Traffic app installation;
(4) the application installation time.

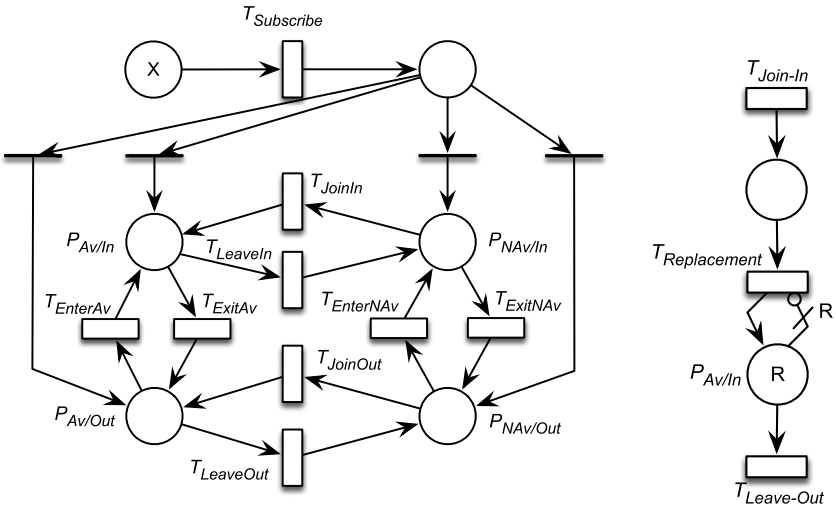
This delay is also captured in the MCSaaS curve of Fig. 14, highlighted in the [0, 10] time unit interval.

In this evaluation we neglected the overhead introduced by the abstraction of sensor resources and registration of infrastructure services, since in [44] we measured metrics pertaining to the abstraction overhead of sensing resources, at the SAaaS level, and we found it could be considered negligible for all purposes. With regard to registration, it can be prospected as a typically one-time (SAaaS-level) operation, so it is not so critical in the long term.

A quantitative, in-depth evaluation on the impact of the churn management in terms of performance cannot be performed due to the limits of the testbed and the preliminary implementation of the MobiCSOS framework. Nonetheless, some general, qualitative observations, based on the experiments conducted, are reported in the following. The impact of resource churn (and subsequent updates of resources) on the performance of the service to be offered (i.e., the MCS application) is rather limited by design, as long as any replacement is application-transparent (as is the case in general for MCSaaS). The overall application behavior may depend on the smoothness of the curve depicting the ratio of available resources to the requested ones over time, as in the case of the traffic monitoring app. Reactivity is thus key mostly when dealing with loss of huge numbers of contributed devices. App deployment time, depending on bandwidth and concurrent (non-blocking) fan-out, is mostly constant, not proportional to the number of devices to be replaced.

8.3.2. Modeling and evaluation

To further highlight the benefits of the MCSaaS approach against the basic MCS one, two simple models representing the paradigms under comparison have been devised and evaluated through Petri nets (Fig. 15). The MCS model of Fig. 15(a) is more complex than the MCSaaS one of Fig. 15(b), since it has to consider the contributor enrollment and churn management, while in the MCSaaS approach this is managed by the SAaaS provider, assuming there are always enough SAaaS contributors available satisfying the app requirements. This assumption is quite realistic considering that, if motivated with specific incentives, the SAaaS



(a) MCS. (b) MCSaaS.

Fig. 15. PN models of the two approaches.

contributor population should be very large, orders of magnitude larger than the one supporting a single app. Furthermore, this availability, paired to the MCSaaS facilities for churn management, should easily allow to always have enough contributors actively engaged by, and contributing to, the app.

We represented a generic MCS application as the pothole mapping or the traffic monitoring ones above described, elaborating data coming from a given area of interest. This way, in the PN of Fig. 15(a), four possible states can be identified for a contributing node, obtained as combinations of contributor availability (Av/NAv) and position on the area of interest (In/Out). A node is thus actively contributing to the MCS app if it is available and in the area of interest ($P_{Av/In}$), while it is not contributing otherwise, i.e., if outside the area or not available ($P_{Av/Out}$, $P_{NAv/In}$, $P_{NAv/Out}$). New contributors may subscribe to the MCS system (through transition $T_{Subscribe}$), and will enter the system in one of the above identified conditions ($P_{Av/In}$, $P_{Av/Out}$, $P_{NAv/In}$, $P_{NAv/Out}$), able to actively support the MCS service only when in the area of interest and available ($P_{Av/In}$). The entry condition is therefore probabilistically specified by assigning corresponding weights to the immediate transitions after the input place: we assigned high (equal) weights (4) to the transitions to $P_{Av/In}$, $P_{Av/Out}$, assuming that 80% of new subscribers are immediately available to contribute, while low weights (1) have been associated with $P_{NAv/In}$ and $P_{NAv/Out}$, meaning that just 20% of new subscribers are not immediately available to contribute. Then, once in the MCS system, subscribers can change their states becoming available/not available or going in/out the area of interest. In the first case, transitions T_{Join*} and T_{Leave*} regulate availability changing, while transitions T_{Enter*} and T_{Exit*} characterize the in/out mobility, respectively.

On the other hand, the MCSaaS system is supported by the SAaaS infrastructure that, as discussed above, we assume is always able to expose the right subset of sensing resources (R), for the scenarios at hand, to the MCS application to be deployed due to a large population of motivated and incentivized contributors. In fact, given large-scale field use of SAaaS-enabled devices, the assumption about availability of a sensor-rich and diverse range of measurable quantities and technologies for sampling may hold, while delegating to the SAaaS components mostly duties related to capability abstraction and discovery. The model thus mainly represents the node churn management, where the replacement of a leaving contributor with one who is available and in the area of interest ($P_{Av/In}$) is stochastically characterized and quantified ($T_{Replacement}$) according to the values obtained by the above experiments. This way $T_{Join-In}$ and $T_{Leave-Out}$ represent the dynamics of active contributors (Av/In) only for the MCSaaS system.

Table 2
Parameters of the PN models (h^{-1}).

$\lambda_{Subscribe}$	λ_{Join}	λ_{Leave}	λ_{Enter}	λ_{Exit}	$\lambda_{Replacement}$	$\lambda_{Join-Enter}$	$\lambda_{Leave-Exit}$
1/24	1/3	1	1/4	1	120	7/12	2

With regard to MCSaaS, at the time the R requested sensing resources are provided ($P_{Av/In-Act}$), the system has N further resources available in the area of interest that could be used as spares in the case of node churn ($P_{Av/In-NAct}$). Other S contributors ($P_{\neg AvIn}$) may join the system or enter the area of interest later on.

In the models we used exponential distributions to stochastically characterize these behaviors, assuming all the related events are effects of random causes and thus randomly distributed. This way two GSPN models are obtained. The distribution parameters have been chosen also based on existing literature [45]. The rates of all the transitions are reported in Table 2. Notice that $\lambda_{Replacement}$ is obtained by the experiments described in the previous section (30 s per replacement). It is important to remark that such rates and the corresponding distributions are marking dependent since each of them represents a single activity or contributor. This is represented in Fig. 15 by a # symbol close to the marking dependent transition.

The evaluation results shown in Fig. 16 demonstrate the effectiveness of the MCSaaS approach. Indeed, considering $R = 10$, $N = 5$ and $S = 20$, the MCSaaS system is able to ensure on the average 9.99 continuously operating nodes, with a probability of 0.989 to provide 10 users, as requested by the MCS. Higher probability can be reached by over-provisioning. The results also highlight the slow dynamics of the MCS system, which requires about 100 days to reach 10 actively contributing nodes on the average, on the basis of about 100 subscribers.

Another interesting metric that allows us to compare the MCS and MCSaaS approaches is the amount of data collected and managed by the system. Assuming a simple model where every node contributes with a constant data rate, e.g., 100 MB/day for a system contributing steadily over 24 h, according to the results of Fig. 16, the total amount of data gathered by the MCS system against the MCSaaS one over the first 10 days is about 50 MB vs 10 GB (a 1:200 ratio in favor of the MCSaaS), 20 GB vs 200 GB (1:10) over 20 days, 125 GB vs 500 GB (1:4) over 50 days, 500GB against 1 TB (1:2) over 100 days, respectively.

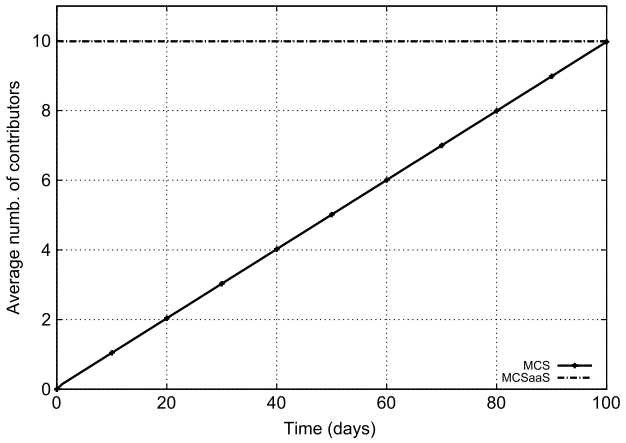


Fig. 16. Comparison between the approaches through PN model evaluation.

9. Conclusions

In this paper, a novel MCS as-a-Service paradigm is proposed which addresses key design and business challenges of current MCS deployment practices. Specifically, a platform for MCS mass deployment is presented that essentially splits the MCS service application and infrastructure into two different levels supporting distinct functions and several business exploitation avenues. The corresponding actors involved, their respective roles and interactions, as well as potential benefits and drawbacks, are identified and discussed. Moreover, a prototype implementation that serves as a proof-of-concept for the conceptual framework is presented, while an evaluation of the soundness of the proposed paradigm is presented using (i) suitable scenarios for proof-of-concept validation and (ii) a modeling approach through Generalized Stochastic Petri Nets. The results demonstrate a greater flexibility of the MCSaaS approach compared to plain MCS, making room for speculation about its possible adoption in commercial contexts, opening up new application domains and unlocking business opportunities. Amid ongoing development efforts we believe we will have the chance to delve further into other interesting use cases and application scenarios, including experimentation on actual devices and heterogeneous platforms.

References

[1] R. Ganti, F. Ye, H. Lei, Mobile crowdsensing: current state and future challenges, IEEE Commun. Mag. 49 (11) (2011) 32–39. http://dx.doi.org/10.1109/MCOM.2011.6069707.
[2] Y. Xiao, P. Simoens, P. Pillai, K. Ha, M. Satyanarayanan, Lowering the barriers to large-scale mobile crowdsensing, in: Proceedings of the 14th Workshop on Mobile Computing Systems and Applications, HotMobile '13, ACM, New York, NY, USA, 2013, pp. 9:1–9:6. http://dx.doi.org/10.1145/2444776.2444789. URL http://doi.acm.org/10.1145/2444776.2444789.
[3] European Commission Definition of a research and innovation policy leveraging cloud computing and iot combination. tender specifications, smart 2013/0037, Tech. rep., European Commission, 2013.
[4] J. Zhou, T. Leppanen, E. Harjula, M. Ylianttila, T. Ojala, C. Yu, H. Jin, L. Yang, Cloudthings: A common architecture for integrating the internet of things with cloud computing, in: 2013 IEEE 17th International Conference on Computer Supported Cooperative Work in Design, CSCWD, 2013, pp. 651–657. http://dx.doi.org/10.1109/CSCWD.2013.6581037.
[5] A. Botta, W. de Donato, V. Persico, A. Pescape, On the integration of cloud computing and internet of things, in: 2014 International Conference on Future Internet of Things and Cloud, FiCloud, 2014, pp. 23–30. http://dx.doi.org/10.1109/FiCloud.2014.14.
[6] S. Distefano, G. Merlino, A. Puliafito, Sensing and actuation as a service: A new development for clouds, in: Proceedings of the 2012 IEEE 11th International Symposium on Network Computing and Applications, NCA '12, IEEE Computer Society, Washington, DC, USA, 2012, pp. 272–275. http://dx.doi.org/10.1109/NCA.2012.38. URL http://dx.doi.org/10.1109/NCA.2012.38.
[7] D. Kartson, G. Balbo, S. Donatelli, G. Franceschinis, G. Conte, Modelling with Generalized Stochastic Petri Nets, John Wiley & Sons, Inc., 1994.
[8] J. Burke, D. Estrin, M. Hansen, A. Parker, N. Ramanathan, S. Reddy, M.B. Srivastava, Participatory sensing, in: In: Workshop on World-Sensor-Web (WSW'06): Mobile Device Centric Sensor Networks and Applications, 2006, pp. 117–134.
[9] N. Lane, E. Miluzzo, H. Lu, D. Peebles, T. Choudhury, A. Campbell, A survey of mobile phone sensing, IEEE Commun. Mag. 48 (9) (2010) 140–150.
[10] B. Guo, Z. Yu, X. Zhou, D. Zhang, From participatory sensing to mobile crowd sensing, in: 2014 IEEE International Conference on Pervasive Computing and Communications Workshops, PERCOM Workshops, 2014, pp. 593–598. http://dx.doi.org/10.1109/PerComW.2014.6815273.
[11] X. Yu, W. Zhang, L. Zhang, V.O. Li, J. Yuan, I. You, Understanding urban dynamics based on pervasive sensing: An experimental study on traffic density and air pollution, Math. Comput. Modelling 58 (56) (2013) 1328–1339. http://dx.doi.org/10.1016/j.mcm.2013.01.002. the Measurement of Undesirable Outputs: Models Development and Empirical Analyses and Advances in mobile, ubiquitous and cognitive computing. URL http://www.sciencedirect.com/science/article/pii/S089571771300006X.
[12] E. Aubry, T. Silverston, A. Lahmadi, O. Festor, Crowdout: A mobile crowdsourcing service for road safety in digital cities, in: 2014 IEEE International Conference on Pervasive Computing and Communications Workshops, PERCOM Workshops, 2014, pp. 86–91. http://dx.doi.org/10.1109/PerComW.2014.6815170.
[13] P. Mohan, V.N. Padmanabhan, R. Ramjee, Nericell: rich monitoring of road and traffic conditions using mobile smartphones, in: Proceedings of the 6th ACM Conference on Embedded Network Sensor Systems, SenSys '08, ACM, New York, NY, USA, 2008, pp. 323–336.
[14] S. Hu, L. Su, H. Liu, H. Wang, T. Abdelzaher, Smartroad: A crowdsourced traffic regulator detection and identification system, in: Proceedings of the 12th International Conference on Information Processing in Sensor Networks, IPSN '13, ACM, New York, NY, USA, 2013, pp. 331–332. http://dx.doi.org/10.1145/2461381.2461433. URL http://doi.acm.org/10.1145/2461381.2461433.
[15] G. Merlino, D. Bruneo, S. Distefano, F. Longo, A. Puliafito, A. Al-Anbuky, A smart city lighting case study on an openstack-powered infrastructure, Sensors 15 (7) (2015) 16314. http://dx.doi.org/10.3390/s150716314. URL http://www.mdpi.com/1424-8220/15/7/16314.
[16] G. Cardone, L. Foschini, P. Bellavista, A. Corradi, C. Borcea, M. Talasila, R. Curtmola, Fostering participaction in smart cities: a geo-social crowdsensing platform, IEEE Commun. Mag. 51 (6) (2013) 112–119. http://dx.doi.org/10.1109/MCOM.2013.6525603.
[17] M. Demirbas, M. Bayir, C. Akcora, Y. Yilmaz, H. Ferhatosmanoglu, Crowdsourced sensing and collaboration using twitter, in: 2010 IEEE International Symposium on a World of Wireless Mobile and Multimedia Networks, WoWMoM, 2010, pp. 1–9.
[18] Y. Chon, N.D. Lane, F. Li, H. Cha, F. Zhao, Automatically characterizing places with opportunistic crowdsensing using smartphones, in: Proceedings of the 2012 ACM Conference on Ubiquitous Computing, UbiComp '12, ACM, New York, NY, USA, 2012, pp. 481–490.
[19] S.B. Eisenman, E. Miluzzo, N.D. Lane, R.A. Peterson, G.-S. Ahn, A.T. Campbell, Bikenet: A mobile sensing system for cyclist experience mapping, ACM Trans. Sens. Netw. 6 (1) (2010) 6:1–6:39.
[20] X. Chen, E. Santos-Neto, M. Ripeanu, Crowdsourcing for on-street smart parking, in: Proceedings of the Second ACM International Symposium on Design and Analysis of Intelligent Vehicular Networks and Applications, DIVANet '12, ACM, New York, NY, USA, 2012, pp. 1–8.
[21] M.-R. Ra, B. Liu, T.F. La Porta, R. Govindan, Medusa: a programming framework for crowd-sensing applications, in: Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services, MobiSys '12, ACM, New York, NY, USA, 2012, pp. 337–350.
[22] Y. Xiao, P. Simoens, P. Pillai, K. Ha, M. Satyanarayanan, Lowering the barriers to large-scale mobile crowdsensing, in: Proceedings of the 14th Workshop on Mobile Computing Systems and Applications, HotMobile '13, ACM, New York, NY, USA, 2013, pp. 9:1–9:6.
[23] C. Cornelius, A. Kapadia, D. Kotz, D. Peebles, M. Shin, N. Triandopoulos, Anonysense: Privacy-aware people-centric sensing, in: Proceedings of the 6th International Conference on Mobile Systems, Applications, and Services, MobiSys '08, ACM, New York, NY, USA, 2008, pp. 211–224. http://dx.doi.org/10.1145/1378600.1378624. URL http://doi.acm.org/10.1145/1378600.1378624.
[24] M.-R. Ra, B. Liu, T.F. La Porta, R. Govindan, Medusa: A programming framework for crowd-sensing applications, in: Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services, ACM, 2012, pp. 337–350.
[25] N. Brouwers, K. Langendoen, Pogo, a middleware for mobile phone sensing, in: Proceedings of the 13th International Middleware Conference, Springer-Verlag New York, Inc., 2012, pp. 21–40.
[26] X. Hu, T. Chu, H. Chan, V. Leung, Vita: A crowdsensing-oriented mobile cyber-physical system, IEEE Trans. Emerg. Top. Comput. 1 (1) (2013) 148–165. http://dx.doi.org/10.1109/TETC.2013.2273359.
[27] T. Das, P. Mohan, V.N. Padmanabhan, R. Ramjee, A. Sharma, Prism: platform for remote sensing using smartphones, in: Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services, ACM, 2010, pp. 63–76.
[28] I. Fette, A. Melnikov, The WebSocket Protocol, RFC 6455, RFC Editor, December 2011. URLhttp://www.rfc-editor.org/rfc/rfc6455.txt.
[29] T. Metsch, A. Edmonds, R. Nyrén, A. Papaspyrou, Open cloud computing interface–core, in: Open Grid Forum, OCCI-WG, Specification Document, 2010. Available at: http://forge.gridforum.org/sf/go/doc16161.
[30] O.T.T. Committee, Topology and Orchestration Specification for Cloud Applications, Tech. Rep., OASIS, URL http://docs.oasis-open.org/tosca/TOSCA/v1.0/os/TOSCA-v1.0-os.html.
[31] R. Mordani, S.W. Chan, Java Servlet Specification, Sun Microsystems Inc., version 3.
[32] Apache Tomcat. URL http://tomcat.apache.org/tomcat-8.0-doc/index.html.
[33] A. Developers, Google cloud messaging for android.
[34] Openstack documentation. URL http://docs.openstack.org.
[35] Jenkins: An extensible open source continuous integration server [cited June 2015]. URL https://jenkins-ci.org/.

[36] AllJoyn. URL http://allseenalliance.org.
[37] PoliMi POLICLOUD [cited Dec 2014]. URL http://policloud.polimi.it/.
[38] Waze: Free gps navigation with turn by turn [cited Dec. 2014]. URL https://www.waze.com/.
[39] R. Long Cheu, C. Xie, D.-H. Lee, Probe vehicle population and sample size for arterial speed estimation, Compu.-Aided Civil Infrastruct. Eng. 17 (1) (2002) 53–60.
[40] W.F. Adams, Road traffic considered as a random series (includes plates), J. ICE 4 (1) (1936) 121–130.
[41] Android open source project [cited Dec. 2014]. URL http://source.android.com/.
[42] Genymotion. URL https://cloud.genymotion.com/page/doc/.
[43] Android Debug Bridge. URL http://developer.android.com/tools/help/adb.html.
[44] S. Distefano, G. Merlino, A. Puliafito, A utility paradigm for iot: The sensing cloud, Pervasive Mobile Comput. 20 (0) (2015) 127–144. http://dx.doi.org/10.1016/j.pmcj.2014.09.006. URL http://www.sciencedirect.com/science/article/pii/S157411921400159X.
[45] S. Choi, M. Baik, C. Hwang, J. Gil, H. Yu, Volunteer availability based fault tolerant scheduling mechanism in desktop grid computing environment, in: Third IEEE International Symposium on Network Computing and Applications, 2004. Proceedings, NCA 2004, IEEE, 2004, pp. 366–371.

Giovanni Merlino is an Electronics Engineer (University of Messina, Italy). He has participated in Italian Grid Initiative-led activities, and since 2012 is a Research Fellow at University of Messina, meanwhile also participating in an international Ph.D. programme at University of Catania. His research interests include distributed systems with particular emphasis on virtualization, Grid and Cloud paradigms, ad-hoc and sensor networks, mobile platforms. He has been involved in national and international research projects.

Stamatis Arkoulis received the BSc. degree in Computer Science and the M.Sc. in Information Systems both from the Athens University of Economics and Business, Greece, in 2007 and 2009 respectively. He received his Ph.D. degree in Electrical and Computer Engineering National Technical University of Athens (NTUA), Greece in 2014. His research interests span the area of modeling and performance evaluation of computer and communication networks, resource reconfiguration in Cognitive Radio networks and optimization modeling and design.

Salvatore Distefano is an assistant professor at Politecnico di Milano. His main research interests include non-Markovian modeling; performance and reliability evaluation; dependability; Quality of Service/Experience; Service Level Agreement; Parallel and Distributed Computing, Grid, Cloud, Autonomic, Volunteer, Crowd Computing; Big Data; Software and Service Engineering. During his research activity, he has contributed in the development of several tools such as WebSPN, ArgoPerformance and GS3. He has been involved in several national and international research projects. He is author and co-author of more than 100 scientific papers. He is member of international conference committees and he is in the editorial boards of the International Journal of Performability Engineering, Journal of Cloud Computing, International Journal of Engineering and Industries, International Journal of Big Data, International Journal of Computer Science & Information Technology Applications. He also acted as guest editors for special issues of the Journal of Risk and Reliability, Journal of Performability Engineering, ACM Performance Evaluation Review and IEEE Transactions on Dependable and Secure Computing.

Chrysa Papagianni is a Senior R&D Associate at the Network Management and Optimal Design Laboratory (NETMODE), National Technical University of Athens (NTUA), Athens, Greece, since October 2010. She obtained her Diploma and the Ph.D. degree in Electrical and Computer Engineering, both from NTUA in 2003 and 2009 respectively. Her research interests lie in the area of computer networks with emphasis on cloud computing, virtualization, network optimization and management, and service provisioning.

Antonio Puliafito is a full professor of computer engineering at the University of Messina, Italy. His interests include parallel and distributed systems, wireless technologies, GRID and Cloud computing. He is also interested in the performance evaluation of such systems. He participated and currently participate to several Italian an European research projects on distributed computing, cyber physical systems and smart cities.

Symeon Papavassiliou is a Professor with the Faculty of Electrical and Computer Engineering Department, National Technical University of Athens. He received the Diploma in Electrical and Computer Engineering from the National Technical University of Athens, Greece, in 1990, and the MSc and Ph.D. degrees in Electrical Engineering from Polytechnic University, Brooklyn, NY, USA, in 1992 and 1996 respectively. From 1995 to 1999, Dr. Papavassiliou was a senior technical staff member at AT&T Laboratories in Middletown, New Jersey, and in August 1999 he joined the Electrical and Computer Engineering Department at the New Jersey Institute of Technology (NJIT), where he was an Associate Professor until 2004. Dr. Papavassiliou was awarded the Best Paper Award in IEEE INFOCOM '94, the AT&T Division Recognition and Achievement Award in 1997, the National Science Foundation (NSF) Career Award in 2003, and the Best Paper Award in IEEE WCNC 2012. Dr. Papavassiliou has an established record of publications in his field of expertise, with more than two hundred technical journal and conference published papers. His main research interests lie in the area of computer and communication networks with emphasis on wireless communications, resource allocation and optimization, network virtualization and performance analysis and evaluation of stochastic systems.